

Name: Dau Vu Tuan Khoi

ID: 105709548

C4 Report

I. Code Explanation and Research Effort

To eliminate the need to manually construct the Deep Learning network by explicitly adding hard-coded layers, I implemented the *build_dynamic_model* function to dynamically take the number of layers, layer sizes, and layer names as inputs. This process required internet research to address specific technical challenges:

1. Dynamic Keras Layer Instantiation:

Instead of using the *eval()* function (which poses security risks and is generally discouraged in Python), I created a mapping dictionary: *layer_dict = ('LSTM': LSTM, 'RNN': SimpleRNN, 'GRU': GRU)*. Assigning *SelectedLayer = layer_dict[layer_name]* allows the program to directly invoke the Keras class based on the provided string argument. This approach was adapted from Python Design Patterns documentation and StackOverflow discussions regarding Keras code optimization.

2. Automated return_sequences Parameter Control:

The logic *is_last_layer = (i == num_layers - 1)* paired with *return_sequences=not is_last_layer* is critical. When stacking recurrent layers (such as LSTM or GRU), all intermediate layers must return a 3D sequence (True) so the subsequent layer can process it. However, the final recurrent layer right before the Output Dense layer must flatten the data into a 2D array (False). The code dynamically mismatch errors.

II. Summary of Experiment Results

Below is a brached summary of the various hyperparameter experiments and different configurations of DL models tested for the stock time-series prediction task:

MODEL TRAINING EXPERIMENT RESULTS (Hyperparameter Configurations)

1. LSTM (Long Short-Term Memory) Architecture – Baseline

Configuration: 3 layers, 50 units/layer, 25 epochs, batch size 32

Evaluation: High stability; the prediction curve tracks the moving average (Mid) trend closely. Loss converges slowly but steadily

Configuration: 4 layers, 100 units/layer, 50 epochs, batch size 64

Evaluation: Significantly longer training time. Shows signs of slight overfitting near the end of the cycle because the architecture is too deep for the current dataset volume.

2. GRU (Gated Recurrent Unit) Architecture – Optimization

Configuration: 3 layers, 50 units/layer, 25 epochs, batch size 32

Evaluation: Training (convergence) speed is approximately 20-30% faster than LSTM due to having fewer computational gates. The error margin (Loss) is comparable to LSTM.

Configuration: Cone structure (3 layers: 128 -> 64 -> 32 units), 30 epochs, batch size 16

Evaluation: A smaller batch size makes the model more responsive to short-term price fluctuations. This configuration achieved the lowest Loss score among all tested versions.

3. Simple RNN (Recurrent Neural Network) Architecture – Baseline Limit Test

Configuration: 3 layers, 50 units/layer, 25 epochs, batch size 32

Evaluation: Poorest performance. Highly affected by the vanishing gradient problem. The model essentially just drags the previous day's value forward, creating a delayed straight line without genuinely predicting future trends.